



Σχεδιασμός Ενσωματωμένων Συστημάτων

9ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

5η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

Άσκηση 1	2
Άσκηση 2	4
Άσκηση 3	6

Άσκηση 1

Στην άσκηση αυτή γράφουμε σε assembly του επεξεργαστή ARM ένα πρόγραμμα που περιλαμβάνει έως 32 χαρακτήρες και μετατρέπει τα κεφαλαία σε μικρά και αντίστροφα, στα ψηφία πραγματοποιεί την παρακάτω πράξη και αφήνει όλους τους υπόλοιπους χαρακτήρες αμετάβλητους.

Το πρόγραμμα είναι συνεχούς λειτουργίας, τερματίζει όταν δεχτεί είσοδο αποτελούμενη μόνο από 'q' ή 'Q' και σε περίπτωση που δεχτεί περισσότερους από 32 χαρακτήρες απορρίπτει τους υπόλοιπους.

Ο κώδικας σε assembly για το πρόγραμμα αυτό βρίσκεται στο αρχείο **ask1.s**. Το πρόγραμμα αποτελείται από ένα κυρίως σώμα (main), στο οποίο τυπώνεται το prompt στον user, λαμβάνεται η είσοδος, προσέχοντας να απορρίψουμε τυχόν περισσότερους από 32 χαρακτήρες. Σημειώνουμε σε αυτό το σημείο ότι έχει υλοποιηθεί με τρόπο ώστε να μπορεί στο string να δεχτεί και spaces. Στην συνέχεια γίνεται έλεγχος για το αν έχουμε μόνο q ή Q και έξοδος ή αν προχωράει το πρόγραμμά μας κανονικά. Σημειώνεται ότι για τον έλεγχο του q όπως και παρακάτω στον κώδικα για έλεγχο γραμμάτων χρησιμοποιείται trick για να γλιτώσουμε έναν έλεγχο, το οποίο μετατρέπει όλα τα γράμματα σε lowercase (παρακάτω χρειαζόμαστε 1 παραπάνω register να διατηρεί το uppercase σε περίπτωση που ήταν):

Παράδειγμα για το trick:

'D' = 0x44 = 0100 0100 'd' = 0x64 = 0110 0100 mask = 0x20 = 0010 0000

Παρατηρούμε ότι μπορούμε με πράξεις bic και orr να χρησιμοποιήσουμε μια μάσκα για έλεγχο για uppercase - lowercase, αλλά και μετατροπή από τον έναν τύπο στον άλλον. Σημειώνουμε ότι έχουμε παρατηρήσει πρώτα ότι ισχύει για όλα τα γράμματα πριν το εφαρμόσουμε.

Η swap function έχει ένα σώμα 25 εντολών assembly στο οποίο έχουμε προσπαθήσει να εκμεταλλευτούμε τόσο την σειρά των labels (για τα διαφορετικά τμήματα κώδικα), όσο και το παραπάνω trick για να γλιτώσουμε branch εντολές. Γίνεται έλεγχος για null character αρχικά, στην συνέχεια για γράμμα πραγματοποιώντας την αντίστοιχη μετατροπή αν έχουμε γράμμα. Σε άλλη περίπτωση γίνεται έλεγχος για αριθμό και σε περίπτωση που δεν έχουμε αριθμό αφήνουμε τον χαρακτήρα ως έχει. Σε περίπτωση αριθμού παρατηρούμε ότι ουσιαστικά η πράξη που κάνουμε για ψηφίο είναι:

$$new_digit = (old_digit + 5) mod 10$$

Την παραπάνω πράξη μπορούμε να την κάνουμε με μια πρόσθεση με 5 και έλεγχο αν ξεπεράσαμε τον ASCII κωδικό του 9 για να αφαιρέσουμε 10. Σημειώνεται ότι δεν είναι ανάγκη να μετατρέψουμε τον αριθμό από ASCII και μετά ξανά σε ASCII για να γλιτώσουμε όσο το δυνατόν περισσότερες εντολές.

Ο τελικός απολογισμός της swap από το push στην στοίβα (μετρώντας το) έως και το pop και την επιστροφή στην main είναι 25 εντολές assembly.

Παρακάτω φαίνεται η εκτέλεση του κώδικα μας στο παράδειγμα της εκφώνησης, αλλά και σε κάποια cases που επιλέγουμε για να ελέγξουμε την ορθή λειτουργία του προγράμματος μας:

```

root@debian-armel:~# ./ask1
Input a string of up to 32 chars long:##asd123$X8B5mnjL9!
Result is: ##ASD678$x3b0MNJl4!
Input a string of up to 32 chars long:1234567890
Result is: 6789012345
Input a string of up to 32 chars long:abcABCxyzXYZ
Result is: ABCabcXYZxyz
Input a string of up to 32 chars long:!@##%$^&*(a1
Result is: !@##%$^&*(A6
Input a string of up to 32 chars long:::qqQQ
Result is: ::QQqq
Input a string of up to 32 chars long:qaa
Result is: QAA

```

Figure 1: Test examples for assembly programm

```

root@debian-armel:~# ./ask1
Input a string of up to 32 chars long:qaa
Result is: QAA
Input a string of up to 32 chars long:q
Exiting...
root@debian-armel:~# ./ask1
Input a string of up to 32 chars long:Q!
Result is: q!
Input a string of up to 32 chars long:Q
Exiting...
root@debian-armel:~# _

```

Figure 2: Testing for correct termination

```

root@debian-armel:~# ./ask1
Input a string of up to 32 chars long:This text is exactly 32 characte
Result is: THIS TEXT IS EXACTLY 87 CHARACTE
Input a string of up to 32 chars long:This text is more than 32 characters!
Result is: THIS TEXT IS MORE THAN 87 CHARAC
Input a string of up to 32 chars long:q
Exiting...
root@debian-armel:~#

```

Figure 3: Tests for 32 and more than 32 characters

Στα παραπάνω παραδείγματα, αρχικά, παρατηρούμε την ορθή λειτουργία του προγράμματός μας για απλά παραδείγματα, όπως αυτό που υπάρχει στην εκφώνηση της άσκησης και παραδείγματα με τα πρώτα και τελευταία γράμματα, με όλους τους αριθμούς και ειδικούς χαρακτήρες. Στο 2ο παράδειγμα ελέγχουμε ότι έχουμε τερματισμό μόνο όταν έχουμε 'q' ή 'Q' μόνο και όχι σε κάποιον άλλον συνδυασμό. Τέλος, στο 3ο παράδειγμα ελέγχουμε ότι δεν υπάρχει πρόβλημα στο πρόγραμμά μας όταν έχουμε input ακριβώς 32 χαρακτήρων ή περισσότερων από 32 χαρακτήρες.

Άσκηση 2

Σκοπός του δεύτερου ερωτήματος είναι η υλοποίηση επικοινωνίας μεταξύ ενός host και ενός guest μηχανήματος μέσω σειριακής θύρας, σε περιβάλλον QEMU. Δεδομένου ότι τα περισσότερα σύγχρονα συστήματα δεν διαθέτουν φυσική σειριακή θύρα, η επικοινωνία υλοποιείται μέσω εικονικής σειριακής θύρας, αξιοποιώντας τον μηχανισμό **pseudoterminal (PTY)** που παρέχει το λειτουργικό σύστημα Linux. Η επικοινωνία πραγματοποιείται μεταξύ ενός προγράμματος σε C στο host μηχανήμα και ενός προγράμματος σε ARM assembly στο guest μηχανήμα, τα οποία ανταλλάσσουν δεδομένα μέσω της εικονικής σειριακής θύρας.

Το QEMU παρέχει δύο εναλλακτικές μεθόδους για την υποστήριξη εικονικής σειριακής θύρας:

1. αυτόματη δημιουργία pseudoterminal με χρήση της επιλογής `-serial pty`,
2. χειροκίνητη δημιουργία pseudoterminal στο host μηχανήμα μέσω ανοίγματος της συσκευής `/dev/ptmx` και παροχή του αντίστοιχου αρχείου `/dev/pts/X` ως όρισμα στο QEMU.

Στο ερώτημα αυτό επιλέξαμε τη δεύτερη μέθοδο, καθώς προσφέρει μεγαλύτερο έλεγχο στη διαδικασία δημιουργίας και διαχείρισης της σειριακής επικοινωνίας, καθώς και τη δυνατότητα φιλτραρίσματος ανεπιθύμητων δεδομένων κατά την εκκίνηση του συστήματος. Στη συνέχεια παρουσιάζουμε αναλυτικά τι κάναμε για την υλοποίηση του host και του guest αντιστοιχία.

Στο host μηχανήμα, το πρόγραμμα ανοίγει τη συσκευή `/dev/ptmx`, γεγονός που οδηγεί αυτόματα στη δημιουργία ενός ζεύγους pseudoterminal (*master-slave*). Το slave άκρο του pseudoterminal εμφανίζεται ως αρχείο στη διαδρομή `/dev/pts/X`, το οποίο χρησιμοποιείται ως όρισμα κατά την εκκίνηση του QEMU:

```
-serial /dev/pts/X
```

Με τον τρόπο αυτό το host πρόγραμμα επικοινωνεί μέσω του master PTY και το guest μηχανήμα αντιλαμβάνεται το slave PTY ως σειριακή συσκευή `/dev/ttyAMA0`. Η σύνδεση αυτή επιτρέπει αμφίδρομη επικοινωνία μεταξύ host και guest, προσομοιώνοντας τη λειτουργία πραγματικής σειριακής θύρας.

Για την αξιόπιστη ανταλλαγή δεδομένων, η σειριακή θύρα στο host παραμετροποιείται σε *raw mode* με χρήση της βιβλιοθήκης `termios`. Με τον τρόπο αυτό απενεργοποιούνται το `canonical mode`, το `echo` χαρακτήρων, οι αυτόματες μετατροπές χαρακτήρων (`CR/LF`) και η επεξεργασία εξόδου. Η παραμετροποίηση αυτή διασφαλίζει ότι η επικοινωνία πραγματοποιείται ανά byte, χωρίς παρεμβάσεις από το λειτουργικό σύστημα. Στην πλευρά του guest από την άλλη, η σειριακή θύρα ανοίγει και χρησιμοποιείται μέσω άμεσων Linux system calls (`open`, `read`, `write`, `ioctl`) σε ARM assembly. Υλοποιείται επίσης και παραμετροποίηση της σειριακής θύρας μέσω `ioctl` και της δομής `struct termios`, ώστε να επιτευχθεί συμβατή raw λειτουργία, όπως και στον host. Αξίζει ακόμα να σημειώσουμε ότι στο guest σύστημα, η σειριακή συσκευή `/dev/ttyAMA0` μπορεί να χρησιμοποιείται από το πρόγραμμα `getty`, το οποίο εκτελείται για την παροχή login μέσω σειριακής κονσόλας. Η ταυτόχρονη χρήση της ίδιας συσκευής από το πρόγραμμα της άσκησης μπορεί να προκαλέσει παρεμβολές ή ανεπιθύμητη συμπεριφορά. Για τον λόγο αυτό, απενεργοποιήθηκε η εκτέλεση του `getty` σχολιάζοντας τη γραμμή:

```
T0:23:respawn:/sbin/getty -L ttyAMA0 9600 vt100
```

στο αρχείο `/etc/inittab`, και ακολούθησε επανεκκίνηση του guest μηχανήματος, σύμφωνα με τις οδηγίες της εκφώνησης.

Κατά την πρώτη χρήση της εφαρμογής παρατηρήσαμε ότι υπάρχουν μηνύματα εκκίνησης του kernel στη σειριακή θύρα. Έτσι, το πρόγραμμα στο host διαβάσει και αγνοεί τα δεδομένα αυτά κατά την εκκίνηση, καθαρίζοντας το buffer της σειριακής επικοινωνίας πριν ξεκινήσει η κανονική ανταλλαγή δεδομένων με το guest. Κατά την κανονική ανταλλαγή δεδομένων ακολουθούμε την εξής διαδικασία:

Το πρόγραμμα στο host μηχανήμα δέχεται ως είσοδο από τον χρήστη ένα string μεγέθους έως 64 χαρακτήρων (χρήση της `read`) και αποστέλλει το string μέσω της εικονικής σειριακής θύρας στο guest (χρήση της `write`). Το πρόγραμμα στο guest μηχανήμα διαβάζει το string από τη σειριακή θύρα (`read`), υπολογίζει τον χαρακτήρα με τη μεγαλύτερη συχνότητα εμφάνισης, εξαιρώντας τον κενό χαρακτήρα (ASCII 32) και σε περίπτωση ισοπαλίας επιλέγει τον χαρακτήρα με τον μικρότερο ASCII κωδικό. Τέλος, αποστέλλει στον host τον χαρακτήρα και το πλήθος εμφανίσεών του (`write`). Η διαδικασία αυτή επαναλαμβάνεται συνεχώς, επιτρέποντας πολλαπλές αλληλεπιδράσεις μεταξύ host και guest και τερματίζει όταν δεχτεί τον χαρακτήρα Q (εφαρμόσαμε την ίδια λογική με την πρώτη άσκηση). Σημειώνουμε ότι μαζί με την αναφορά παρέχονται και οι κώδικες τόσο για τον host (`communication.c`) όσο και για τον guest (`guest_freq.s`) μαζί με τα απαραίτητα σχόλια για την κατανόησή τους. Παρακάτω παρουσιάζουμε ένα παράδειγμα εκτέλεσης της επικοινωνίας, όπου φαίνεται η πλήρης λειτουργικότητα του συστήματός μας:

```
=====
Use this argument for QEMU:

    -serial /dev/pts/1
=====

Start QEMU with that serial argument, THEN press Enter here.

(Ignored 51 bytes of startup logs)
PTY buffer cleared. Ready for communication.

Please give a string to send to host: Hello host! Nice to meet you
RAW reply bytes: 65 20 34
The most frequent character is "e"
and it appeared 4 times.

Please give a string to send to host: sssttt
RAW reply bytes: 73 20 33
The most frequent character is "s"
and it appeared 3 times.

Please give a string to send to host: Q
Exiting communication.
```

Figure 4: Test example for PTY communication

Στο παραπάνω παράδειγμα φαίνεται ενδεικτική εκτέλεση της εφαρμογής επικοινωνίας host-guest μέσω εικονικής σειριακής θύρας. Στο πρόγραμμα ο host δημιουργεί pseudoterminal, εμφανίζει το αντίστοιχο αρχείο `/dev/pts/1` και ζητά την εκκίνηση του QEMU με το συγκεκριμένο όρισμα σειριακής θύρας. Κατά την εκκίνηση της επικοινωνίας, το host πρόγραμμα διαβάζει και αγνοεί τα αρχικά δεδομένα της σειριακής θύρας (μηνύματα εκκίνησης του guest), καθαρίζοντας το buffer πριν την ανταλλαγή δεδομένων. Στη συνέχεια, ο χρήστης εισάγει ένα string, το οποίο αποστέλλεται μέσω της σειριακής θύρας στο guest. Το host αποστέλλει strings στο guest, το οποίο υπολογίζει τον χαρακτήρα με τη μέγιστη συχνότητα εμφάνισης (με επίλυση ισοπαλιών βάσει ASCII) και επιστρέφει το αποτέλεσμα, επιβεβαιώνοντας την ορθή λειτουργία της επικοινωνίας.

Άσκηση 3

Στην άσκηση αυτήν θα συνδυάσουμε κώδικα γραμμένο σε C με συναρτήσεις υλοποιημένες σε assembly του επεξεργαστή ARM. Ο στόχος μας είναι να αντικαταστήσουμε στον κώδικα σε C που δίνεται τις συναρτήσεις `strlen`, `strcpy`, `strcat` και `strcmp` της βιβλιοθήκης `string.h` με δικές μας υλοποιήσεις σε assembly. Στον κώδικα σε C η μόνη αλλαγή που πραγματοποιούμε είναι να δηλώσουμε ως `extern` τις συναρτήσεις που έχουμε υλοποιήσει και να κάνουμε σχόλιο την γραμμή που κάνει `include` την σχετική βιβλιοθήκη της C.

Οι κώδικες για τις συναρτήσεις βρίσκονται στα αρχεία `{strlen, strcpy, strcat, strcmp}.s` τα οποία υποβάλλονται μαζί με την αναφορά της άσκησης και παρακάτω εξηγείται για κάθε συνάρτηση η λογική πίσω από την υλοποίησή της:

- `strlen`

Η λειτουργία της συνάρτησης αυτής είναι να μετράει το μήκος ενός string. Ως όρισμα δέχεται μια διεύθυνση για τον 1ο χαρακτήρα του string και επιστρέφει τον αριθμό χαρακτήρων του string.

Στον κώδικά μας διατηρούμε την διεύθυνση σε έναν καταχωρητή και κάνουμε `load` με `post increment` μέχρι ο χαρακτήρας που έχουμε να είναι `'\0'`. Όταν βρούμε το τέλος του string διατηρούμε στον καταχωρητή `r0` την διαφορά της διεύθυνσης που προχωράει μείον την αρχική, προσέχοντας να αφαιρέσουμε ακόμα 1, καθώς λόγω του `post-increment` έχουμε μετρήσει έναν παραπάνω χαρακτήρα. Σημειώνουμε για την λογική με το `post-increment` και την αφαίρεση στο τέλος ότι είναι ο πιο οικονομικός τρόπος από άποψη εντολών και για αυτό επιλέγεται. Στο τέλος της συνάρτησης, καλούμε `bx lr`, όπως και παρακάτω, καθώς δεν χρειάζεται να κάνουμε `push` και `pop` στην στοίβα κάτι, καθώς δεν καλούμε άλλη συνάρτηση και δεν αλλάζουμε τον καταχωρητή `lr`.

- `strcpy`

Η συνάρτηση αυτή δέχεται δύο pointers, έναν `source` και έναν `destination` και αντιγράφει ένα string. Στον κώδικα που υλοποιούμε κάνουμε `load` και `store` από τον έναν στον άλλον (2 εντολές για αυτό) μέχρι να βρούμε το `null termination`, το οποίο πρέπει να έχουμε αντιγράψει και μετά επιστρέφει η συνάρτησή μας. Για το τι επιστρέφει η συνάρτησή μας σημειώνουμε ότι έχουμε χρησιμοποιήσει έναν 2ο καταχωρητή με την αρχική τιμή του pointer, για να μην αλλάξουμε την τιμή του pointer που έχουμε ως είσοδο, ώστε να επιστρέφει σωστά ο `destination pointer`.

- `strcat`

Η συνάρτηση αυτή κάνει concatenate δύο strings σε ένα. Δέχεται τους pointers για τα δύο και επιστρέφει την αρχή του concatenated string. Στον κώδικά μας αρχικά διατρέχουμε το 1ο string για να βρούμε το τέλος του και στην συνέχεια στο τέλος του αντιγράφουμε το 2ο, προσέχοντας να κάνουμε `overwrite` το `'\0'` του 1ου string. Όπως και πριν για το `return` έχουμε διατηρήσει τον καταχωρητή που διατηρεί τον pointer στο 1ο string χωρίς αλλαγή.

- `strcmp`

Η λειτουργία της συνάρτησης αυτής είναι να λαμβάνει ως είσοδο δύο strings και να επιστρέφει 0 αν αυτά είναι ίδια ή μια μη μηδενική τιμή αν δεν είναι ίδια, η οποία εξαρτάται από την διαφορά του 1ου χαρακτήρα ο οποίος δεν είναι ίδιος (στην περίπτωσή μας).

Στην υλοποίησή μας κάνουμε `load` κάθε χαρακτήρα με την σειρά και για τους δύο pointers των strings, ελέγχουμε αν δεν είναι ίδια για να επιστρέψουμε την διαφορά τους και αν τελικά είναι ίδιοι όλοι οι χαρακτήρες και φτάνουμε σε `null terminator` και στα δύο ταυτόχρονα, τότε επιστρέφουμε 0.

Σημειώνουμε για την διαδικασία παραγωγής του εκτελέσιμου στο Makefile το οποίο υποβάλλουμε με την αναφορά ότι η διαδικασία που πρέπει να ακολουθήσουμε είναι πρώτα `compilation` σε object file τόσο του C όσο και του assembly κώδικα και στην συνέχεια να κάνουμε `link` τα object files για να παράξουμε το τελικό εκτελέσιμο `string_manipulation.out`.

Για τον έλεγχο ορθότητας των συναρτήσεών μας ακολουθούμε την παρακάτω διαδικασία:

1. Αρχικά τρέχουμε τον κώδικα σε C με την βιβλιοθήκη string.h και αποθηκεύουμε τα αποτελέσματα ως αρχεία με ίδιο όνομα και suffix _orig στο όνομά τους.
2. Τρέχουμε τον κώδικα χωρίς την βιβλιοθήκη, δηλώνοντας ως extern τις συναρτήσεις μας.
3. Με ένα bash script το οποίο υποβάλλουμε και στην αναφορά ελέγχουμε αν τα αρχεία από τους δύο τρόπους εκτέλεσης ταυτίζονται. Παρατηρούμε το παρακάτω αποτέλεσμα, οπότε συμπεραίνουμε ότι τα αρχεία ταυτίζονται για τις 2 μεθόδους και για τα δύο πιθανά inputs:

```
root@debian-armel:~# ./compare_files.sh
OK: rand_str_input_first_orig.txt_concat_out == rand_str_input_first.txt_concat_out
OK: rand_str_input_first_orig.txt_len_out == rand_str_input_first.txt_len_out
OK: rand_str_input_first_orig.txt_sorted_out == rand_str_input_first.txt_sorted_out
OK: rand_str_input_sec_orig.txt_concat_out == rand_str_input_sec.txt_concat_out
OK: rand_str_input_sec_orig.txt_len_out == rand_str_input_sec.txt_len_out
OK: rand_str_input_sec_orig.txt_sorted_out == rand_str_input_sec.txt_sorted_out
root@debian-armel:~#
```

Figure 5: Results for txt files matching

Παρατηρούμε, λοιπόν, ότι τα αρχεία που παράγει το πρόγραμμά μας χρησιμοποιώντας τις συναρτήσεις γραμμένες σε assembly παράγει ακριβώς τα ίδια αρχεία με αυτά που παράγονται όταν χρησιμοποιούνται οι συναρτήσεις της βιβλιοθήκης string.h, συνεπώς, επιβεβαιώνουμε την ορθή λειτουργία του κώδικά μας.